

```

import { useState, useEffect, useRef, useCallback } from "react";

// ——— CONSTANTS ———
const LAYERS = [
  { id: "cognitive", label: "COGNITIVE", sub: "Human Brain", icon: "◆", color: "#
  { id: "mobile",    label: "MOBILE",    sub: "Cell Network", icon: "●", color: "
  { id: "host",      label: "HOST PC",    sub: "Compute Node", icon: "▣", color: "
];

const PIPELINE_STEPS = [
  { id: "ingest",    label: "INGEST",    desc: "Secure nonprofit data intake", la
  { id: "classify", label: "CLASSIFY",   desc: "Domain + impact taxonomy",      1
  { id: "compress", label: "COMPRESS",   desc: "Edge-ready mobile packaging",   1
  { id: "route",     label: "ROUTE",     desc: "Dispatch vector generation",    1
  { id: "compute",   label: "COMPUTE",   desc: "Full graph resolution",
  { id: "sign",      label: "SIGN",      desc: "Cryptographic attestation",     1
  { id: "flywheel",  label: "FLYWHEEL",  desc: "ZIP emission + node spawn",     1
];

const DOMAINS = [
  "Food Security", "Mental Health", "Education Access", "Clean Water",
  "Shelter", "Climate Resilience", "Medical Aid", "Legal Defense",
  "Youth Empowerment", "Elder Care"
];

const sleep = (ms) => new Promise(r => setTimeout(r, ms));

function generateManifest(org) {
  const ts = new Date().toISOString();
  const id = `NP-${Math.random().toString(36).slice(2,10).toUpperCase()}`;
  return {
    node_id: id,
    spawned_at: ts,
    org_name: org.name,
    mission_hash: btoa(org.mission).slice(0, 16),
    domain: org.domain,
    reach: org.reach,
    layers: ["cognitive", "mobile", "host"],
    dispatch_protocol: "world-engine/v1",
    flywheel_version: "3.3.0",
    is_dispatch_node: true,
    can_spawn: true,
    children: [],
    integrity: `sha256:${Array.from({length:16}, ()=>Math.floor(Math.random()*256)

```

```

    };
}

// ——— NODE PULSE —————
function Pulse({ color, size = 40, active }) {
  return (
    <div style={{ position: "relative", width: size, height: size, display: "flex"
      {active && (
        <>
          <div style={{
            position: "absolute", width: size, height: size, borderRadius: "50%",
            border: `1px solid ${color}`, animation: "pulse-ring 1.4s ease-out in
          }} />
          <div style={{
            position: "absolute", width: size * 0.7, height: size * 0.7, borderRa
            border: `1px solid ${color}`, animation: "pulse-ring 1.4s ease-out in
          }} />
        </>
      )}
    <div style={{
      width: size * 0.4, height: size * 0.4, borderRadius: "50%",
      background: active ? color : "#1a1a2e",
      border: `1px solid ${color}`,
      transition: "background 0.3s",
      boxShadow: active ? `0 0 12px ${color}` : "none"
    }} />
  </div>
  );
}

```

```

// ——— STEP ROW —————
function StepRow({ step, status, layerColor }) {
  const statusIcons = { idle: "o", active: "o", done: "●", error: "x" };
  return (
    <div style={{
      display: "flex", alignItems: "center", gap: 12, padding: "7px 0",
      borderBottom: "1px solid rgba(255,255,255,0.05)",
      opacity: status === "idle" ? 0.4 : 1,
      transition: "opacity 0.3s"
    }}>
      <span style={{
        color: status === "done" ? layerColor : status === "active" ? "#fff" : "#
        fontSize: 13, fontFamily: "monospace", width: 14, textAlign: "center",
        animation: status === "active" ? "blink 0.8s step-end infinite" : "none"
      }}>{statusIcons[status]}</span>
      <span style={{ color: layerColor, fontSize: 10, fontFamily: "'Space Mono',
        {step.label}

```

```

    </span>
    <span style={{ color: "#888", fontSize: 11, fontFamily: "monospace" }}>{ste
  </div>
);
}

```

```

// — NETWORK GRAPH (mini) —————
function NetworkGraph({ nodes }) {
  const canvasRef = useRef(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext("2d");
    const W = canvas.width, H = canvas.height;
    ctx.clearRect(0, 0, W, H);

    if (!nodes.length) return;

    const positions = nodes.map((_, i) => {
      if (i === 0) return { x: W/2, y: H/2 };
      const angle = (i - 1) * (2 * Math.PI / Math.max(nodes.length - 1, 1));
      const r = Math.min(W, H) * 0.35;
      return { x: W/2 + r * Math.cos(angle), y: H/2 + r * Math.sin(angle) };
    });

    // Draw edges
    nodes.forEach((_, i) => {
      if (i === 0) return;
      ctx.beginPath();
      ctx.moveTo(positions[0].x, positions[0].y);
      ctx.lineTo(positions[i].x, positions[i].y);
      ctx.strokeStyle = "rgba(0,255,178,0.2)";
      ctx.lineWidth = 1;
      ctx.stroke();
    });

    // Draw nodes
    nodes.forEach((node, i) => {
      const { x, y } = positions[i];
      const color = i === 0 ? "#00FFB2" : "#FFB200";
      ctx.beginPath();
      ctx.arc(x, y, i === 0 ? 7 : 4, 0, 2*Math.PI);
      ctx.fillStyle = color;
      ctx.fill();
      ctx.shadowColor = color;
      ctx.shadowBlur = 8;
    });
  });
}

```

```

    ctx.fill();
    ctx.shadowBlur = 0;

    ctx.fillStyle = "rgba(255,255,255,0.6)";
    ctx.font = "8px monospace";
    ctx.fillText(node.node_id?.slice(0,6) || "ORIGIN", x + 9, y + 3);
  });
}, [nodes]);

return (
  <canvas ref={canvasRef} width={280} height={180}
    style={{ display: "block", margin: "0 auto" }} />
);
}

// ——— MAIN APP —————
export default function WorldEngine() {
  const [phase, setPhase] = useState("idle"); // idle | form | processing
  const [org, setOrg] = useState({ name: "", mission: "", domain: DOM
  const [stepStatus, setStepStatus] = useState({});
  const [activeStep, setActiveStep] = useState(null);
  const [log, setLog] = useState([]);
  const [manifest, setManifest] = useState(null);
  const [nodes, setNodes] = useState([]);
  const [ticker, setTicker] = useState(0);
  const logRef = useRef(null);

  // Ticker for ambient animations
  useEffect(() => {
    const t = setInterval(() => setTicker(x => x + 1), 1200);
    return () => clearInterval(t);
  }, []);

  useEffect(() => {
    if (logRef.current) logRef.current.scrollTop = logRef.current.scrollHeight;
  }, [log]);

  const addLog = useCallback((msg, color = "#888") => {
    setLog(l => [...l, { msg, color, ts: new Date().toLocaleTimeString("en-US", {
  }, []);

  const runPipeline = useCallback(async () => {
    setPhase("processing");
    setStepStatus({});
    setLog([]);
    addLog("► World Engine v3.3 initializing...", "#00FFB2");
    await sleep(400);

```

```

addLog(`► Organization: ${org.name}`, "#aaa");
addLog(`► Domain: ${org.domain}`, "#aaa");
await sleep(300);

const finalStatus = {};
for (const step of PIPELINE_STEPS) {
  setActiveStep(step.id);
  setStepStatus(s => ({ ...s, [step.id]: "active" }));
  addLog(` ◦ ${step.label}: ${step.desc}`, "#666");
  await sleep(600 + Math.random() * 600);
  finalStatus[step.id] = "done";
  setStepStatus(s => ({ ...s, [step.id]: "done" }));
  addLog(` • ${step.label} complete`, "#00FFB2");
}

setActiveStep(null);
await sleep(400);
addLog("► Generating flywheel manifest...", "#FFB200");
await sleep(500);

const m = generateManifest(org);
setManifest(m);
setNodes(prev => [...prev, m]);
addLog(`► Node spawned: ${m.node_id}`, "#B200FF");
addLog("► ZIP flywheel ready for emission.", "#00FFB2");
addLog("► This node is now a dispatch submodule.", "#FFB200");
addLog("► The world heals.", "#fff");

setPhase("complete");
}, [org, addLog]);

const downloadFlywheel = () => {
  const content = JSON.stringify({ manifest, pipeline: PIPELINE_STEPS, org }, n
  const blob = new Blob([content], { type: "application/json" });
  const a = document.createElement("a");
  a.href = URL.createObjectURL(blob);
  a.download = `flywheel-${manifest.node_id}.json`;
  a.click();
};

const reset = () => {
  setPhase("idle");
  setOrg({ name: "", mission: "", domain: DOMAINS[0], reach: "" });
  setStepStatus({});
  setActiveStep(null);
  setLog([]);
  setManifest(null);

```

```

};

// — Layer color lookup
const layerColor = (layerId) => LAYERS.find(l => l.id === layerId)?.color || "#

return (
  <div style={{
    minHeight: "100vh", background: "#050510",
    fontFamily: "'Space Mono', 'Courier New', monospace",
    color: "#e0e0e0",
    backgroundImage: "radial-gradient(ellipse at 20% 20%, rgba(0,255,178,0.03)
  }}>
  <style>{`
    @import url('https://fonts.googleapis.com/css2?family=Space+Mono:wght@400
    @keyframes pulse-ring { 0% { transform: scale(0.8); opacity:0.8; } 100% {
    @keyframes blink { 50% { opacity:0; } }
    @keyframes scanline { 0%{top:0} 100%{top:100%} }
    @keyframes fadeup { from{opacity:0;transform:translateY(8px)} to{opacity:
    * { box-sizing: border-box; }
    ::-webkit-scrollbar { width: 4px; } ::-webkit-scrollbar-track { backgroun
    ::-webkit-scrollbar-thumb { background: #1e1e3f; }
    input, select { background: #0a0a1a; border: 1px solid #1e1e3f; color: #e
    input:focus, select:focus { border-color: #00FFB2; }
    select option { background: #0a0a1a; }
  `}</style>

  {/* — HEADER */}
  <div style={{ padding: "28px 32px 0", display: "flex", alignItems: "flex-st
    <div>
      <div style={{ fontSize: 10, letterSpacing: 5, color: "#00FFB2", marginB
        <div style={{ fontFamily: "'Syne', sans-serif", fontSize: 28, fontWeigh
          NONPROFIT<br/>FLYWHEEL<br/><span style={{ color: "#00FFB2" }}>DISPATC
        </div>
      <div style={{ fontSize: 10, color: "#555", marginTop: 6, letterSpacing:
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 10, paddingT
      {LAYERS.map((l, i) => (
        <div key={l.id} style={{ display: "flex", alignItems: "center", gap:
          <Pulse color={l.color} size={28} active={
            phase === "processing" && activeStep && PIPELINE_STEPS.find(s=>s.
          } />
        <div>
          <div style={{ fontSize: 9, letterSpacing: 2, color: l.color }}>{l
            <div style={{ fontSize: 9, color: "#444" }}>{l.sub}</div>
          </div>
        </div>
      )})

```

```

    </div>
  </div>

  <div style={{ padding: "20px 32px", display: "grid", gridTemplateColumns: "

    { /* — LEFT: INPUT / PIPELINE */ }

    <div style={{ display: "flex", flexDirection: "column", gap: 16 }}>

      { /* Input Form */ }
      <div style={{ background: "#0a0a1a", border: "1px solid #1e1e3f", paddi
        <div style={{ fontSize: 9, letterSpacing: 3, color: "#555", marginBot
        <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
          <div>
            <div style={{ fontSize: 9, color: "#555", marginBottom: 4, letter
              <input value={org.name} onChange={e=>setOrg(o=>({...o,name:e.targ
                placeholder="Feeding Hope Network" style={{ width: "100%" }}
                disabled={phase === "processing"} />
            </div>
          </div>
          <div>
            <div style={{ fontSize: 9, color: "#555", marginBottom: 4, letter
              <input value={org.mission} onChange={e=>setOrg(o=>({...o,mission:
                placeholder="We eliminate food insecurity in..." style={{ width:
                disabled={phase === "processing"} />
            </div>
          </div>
        <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap:
          <div>
            <div style={{ fontSize: 9, color: "#555", marginBottom: 4, lett
              <select value={org.domain} onChange={e=>setOrg(o=>({...o,domain
                style={{ width: "100%" }} disabled={phase === "processing"}>
                {DOMAINS.map(d => <option key={d}>{d}</option>)}
              </select>
            </div>
          </div>
          <div>
            <div style={{ fontSize: 9, color: "#555", marginBottom: 4, lett
              <input value={org.reach} onChange={e=>setOrg(o=>({...o,reach:e.
                placeholder="12000" style={{ width: "100%" }}
                disabled={phase === "processing"} />
            </div>
          </div>
        </div>
      </div>
    <div style={{ marginTop: 14, display: "flex", gap: 8 }}>
      <button onClick={runPipeline}
        disabled={!org.name || !org.mission || phase === "processing"}
        style={{
          flex: 1, padding: "10px 0", background: org.name && org.mission
            color: org.name && org.mission && phase !== "processing" ? "#05
            border: "none", cursor: org.name && org.mission ? "pointer" : "

```

```

        fontFamily: "'Space Mono', monospace", fontSize: 11, letterSpacing: "all 0.2s"
      }}>
      {phase === "processing" ? "PROCESSING..." : "► GENERATE FLYWHEEL"}
    </button>
    {phase === "complete" && (
      <button onClick={reset} style={{
        padding: "10px 14px", background: "transparent", border: "1px solid #888",
        color: "#888", cursor: "pointer", fontFamily: "'Space Mono', monospace"
      }}>↺</button>
    )}
  </div>
</div>

{/* Pipeline Steps */}
<div style={{ background: "#0a0a1a", border: "1px solid #1e1e3f", padding: 10 }}>
  <div style={{ fontSize: 9, letterSpacing: 3, color: "#555", marginBottom: 10 }}>
    {PIPELINE_STEPS.map(step => (
      <StepRow key={step.id} step={step}
        status={stepStatus[step.id] || "idle"}
        layerColor={layerColor(step.layer)} />
    ))}
  </div>
</div>

{/* — RIGHT: LOG + OUTPUT */}
<div style={{ display: "flex", flexDirection: "column", gap: 16 }}>

  {/* Log terminal */}
  <div style={{
    background: "#030308", border: "1px solid #1e1e3f", padding: 14,
    height: 220, overflow: "hidden", position: "relative"
  }}>
    <div style={{
      fontSize: 9, letterSpacing: 3, color: "#555", marginBottom: 10
    }}>
      <div ref={logRef} style={{ height: 170, overflowY: "auto", display: "flex" }}>
        {log.length === 0 && (
          <div style={{ color: "#333", fontSize: 11 }}>awaiting input...</div>
        )}
        {log.map((entry, i) => (
          <div key={i} style={{ display: "flex", gap: 8, animation: "fadeup 0.5s" }}>
            <span style={{ color: "#333", flexShrink: 0 }}>{entry.ts}</span>
            <span style={{ color: entry.color }}>{entry.msg}</span>
          </div>
        ))}
      </div>
    </div>
  </div>
</div>

```



```

    { /* Network visualization */ }
    <div style={{ background: "#0a0a1a", border: "1px solid #1e1e3f", paddi
      <div style={{ fontSize: 9, letterSpacing: 3, color: "#555", marginBot
        <NetworkGraph nodes={nodes} />
      </div>
    </div>

    { /* Manifest output */ }
    {manifest && (
      <div style={{ background: "#050510", border: "1px solid #00FFB2", pad
        <div style={{ fontSize: 9, letterSpacing: 3, color: "#00FFB2", marg
          <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap:
            [[
              ["NODE ID", manifest.node_id],
              ["DOMAIN", manifest.domain],
              ["PROTOCOL", manifest.dispatch_protocol],
              ["CAN SPAWN", "TRUE"],
              ["INTEGRITY", manifest.integrity?.slice(0,18)+"..."],
              ["VERSION", manifest.flywheel_version],
            ]
            .map(([k,v]) => (
              <div key={k}>
                <div style={{ fontSize: 8, color: "#555", letterSpacing: 2 }}
                <div style={{ fontSize: 10, color: "#e0e0e0" }}>{v}</div>
              </div>
            ))
          </div>
        <button onClick={downloadFlywheel} style={{
          width: "100%", padding: "9px 0", background: "transparent",
          border: "1px solid #00FFB2", color: "#00FFB2",
          fontFamily: "'Space Mono', monospace", fontSize: 10, letterSpacing: 1,
          cursor: "pointer", transition: "all 0.2s"
        }}
        onMouseEnter={e=>{ e.target.style.background="#00FFB2"; e.target.st
        onMouseLeave={e=>{ e.target.style.background="transparent"; e.targe
          ↓ EMIT FLYWHEEL ZIP
        </button>
        <div style={{ marginTop: 10, fontSize: 9, color: "#444", textAlign:
          THIS NODE IS NOW A DISPATCH SUBMODULE
        </div>
      </div>
    )}
  </div>
</div>

    { /* — FOOTER / DOCTRINE */ }
    <div style={{ padding: "0 32px 28px", maxWidth: 900, margin: "0 auto" }}>
      <div style={{ borderTop: "1px solid #1e1e3f", paddingTop: 16, display: "g
        [[

```

```
["MODULAR FUNCTION", "Each flywheel is a self-contained dispatch node
["3x3 WORLD ENGINE", "Cognitive layer classifies. Mobile layer compre
["THE FLYWHEEL", "Each ZIP output enables a new nonprofit to become a
].map(([title, body]) => (
  <div key={title}>
    <div style={{ fontSize: 9, color: "#00FFB2", letterSpacing: 3, marg
    <div style={{ fontSize: 10, color: "#555", lineHeight: 1.6 }}>{body
  </div>
  )})
</div>
</div>
</div>
);
}
```